

Spark's Basic Architecture

- Apache Spark is an open-source, distributed computing framework designed for fast, large-scale data processing.
- Spark was built to overcome the limitations of Hadoop MapReduce, which was slow due to its heavy reliance on reading and writing data to disk between each processing step.
- Spark solved this by introducing in-memory computation, where intermediate data is kept in RAM rather than written to disk, making it significantly faster for most workloads.

The Master-Worker Model

Spark follows a **master-worker architecture**, which means the entire system is divided into two roles:

- **Master** – One central process that controls and coordinates everything
- **Workers** – Multiple machines that carry out the actual data processing tasks

This separation ensures that the logic of the application and the execution of the application are kept independent of each other, making the system easier to scale and manage.

How the Architecture is Organized

Spark's architecture is organized into three main layers, each with a distinct responsibility:

Layer 1 — The Driver (Master)

The Driver is where the user's application runs. It is responsible for understanding the code, building an execution plan, and breaking that plan into smaller tasks. It acts as the brain of the entire operation it decides what needs to be done and in what order.

Layer 2 — The Cluster Manager (Resource Layer)

The Cluster Manager sits between the Driver and the Worker Nodes. It does not participate in computation — its only job is to manage resources. When the Driver needs executors to run tasks, it requests them from the Cluster Manager, which then allocates the appropriate CPU and memory from the available Worker Nodes.

Layer 3 — The Worker Nodes (Execution Layer)

Worker Nodes are the machines that perform the actual computation. Each Worker Node runs one or more Executors — processes that receive tasks from the Driver, execute them against the data, and report results back.

How They Work Together

When a Spark application is submitted, the following sequence takes place:

1. The Driver starts and initializes the Spark Session and Spark Context
2. The Driver contacts the Cluster Manager and requests resources
3. The Cluster Manager launches Executors on available Worker Nodes
4. The Driver breaks the application into tasks and sends them to the Executors
5. Executors process the data and send results back to the Driver
6. Once all tasks are complete, the Executors are released and the application ends

Core Components

1. CLUSTER MANAGER

The Cluster Manager is responsible for allocating and managing resources across the cluster. It acts as the intermediary between the Driver and the Worker Nodes.

Types of Cluster Managers supported by Spark:

- **Standalone** – Spark's own built-in cluster manager
- **Apache YARN** – Resource manager from the Hadoop ecosystem
- **Apache Mesos** – A general-purpose cluster manager
- **Kubernetes** – A container orchestration platform

Key Responsibilities:

- Allocating CPU and memory resources to applications
- Launching executors on Worker Nodes
- Monitoring the health of nodes in the cluster
- Handling node failures and resource recovery

2. SPARK SESSION

Spark Session is the unified entry point for a Spark application, introduced in Spark 2.0. It consolidates SparkContext, SQLContext, and HiveContext into a single interface, making it easier to interact with Spark.

Key Responsibilities:

- Serving as the single point of entry for all Spark functionality
- Creating and managing DataFrames and Datasets
- Running SQL queries on structured data
- Reading from and writing to various data sources
- Managing Spark configurations and runtime properties

3. SPARK CONTEXT

Spark Context is the original entry point for Spark applications prior to Spark 2.0. It remains the core connection between the Driver program and the cluster. In modern applications, it is accessible through the Spark Session.

Key Responsibilities:

- Establishing a connection between the Driver and the Cluster Manager
- Creating RDDs (Resilient Distributed Datasets)
- Broadcasting variables and accumulating shared data across nodes
- Configuring and managing the execution environment

4. DRIVER NODE

The Driver Node is the central controller of a Spark application. It is the machine where the main program runs and where the Spark Session and Spark Context live.

Key Responsibilities:

- Converting user code into a logical execution plan (DAG)
- Breaking the DAG into stages and tasks and scheduling them
- Communicating with the Cluster Manager to request resources
- Monitoring the progress of tasks running on Worker Nodes
- Collecting final results and returning them to the user

The Driver Node is the brain of the Spark application. If the Driver fails, the entire application fails.

5. WORKER NODE

Worker Nodes are the machines in the cluster that perform the actual computation. They receive tasks from the Driver, execute them, and report results back.

Each Worker Node contains:

- **Executor** – A JVM process launched on the worker to run tasks
- **Task Slots** – Individual threads within an executor for parallel execution
- **Cache / Storage** – Memory or disk space for caching intermediate data

Key Responsibilities:

- Executing tasks assigned by the Driver
- Storing intermediate and cached data (RDDs, DataFrames)
- Reporting task status and metrics back to the Driver
- Running multiple tasks in parallel based on available resources

DEPLOYMENT MODES

CLIENT MODE

In Client Mode, the Driver program runs on the machine that submits the Spark job, which is outside the cluster. The client machine must remain connected throughout the application's lifetime.

Characteristics:

- The Driver is located on the local/client machine
- Logs and output are directly visible on the client
- Best suited for interactive sessions and debugging
- The client machine must stay alive for the job to continue
- Network latency can be higher since results travel back to the client

CLUSTER MODE

In Cluster Mode, the Driver program runs inside the cluster itself, on one of the cluster nodes. Once the job is submitted, the client machine can disconnect.

Characteristics:

- The Driver is located inside the cluster
- Logs are accessible through the cluster's web UI
- Best suited for production workloads
- More fault-tolerant since everything runs within the cluster
- Lower network latency as data stays within the cluster